Department of Computer Science & Engineering

Software Engineering
UE24CS342AB2

SE Lab 4: VibeCoding — Battleship vs AI Submission

Name: Devansh Gupta	SRN: PES1UG24CS145	Section: C

Section 1

Section 1 - Original defect and reproduction

Task: Inspect the original modules and reproduce the coordinate and turn-state defects before changing the game.

Original behavior: The baseline revision was 68e3f33. Board placement and player input used zero-based tuples, but AI.choose returned a one-based string. The game parsed that string without subtracting one and checked the wrong cell. It also never called player.fire for incoming AI shots.

Coordinate trace: For AI selection (1, 1), the original AI returned "2,2" and the game tested (2, 2): an occupied cell became a miss. For selection (0, 0), it returned "1,1" and tested (1, 1): an empty cell became a hit. Both cases were reproduced from the preserved original source.

Gameplay evidence: The before video was captured before editing. Firing at 3,3 twice produced HIT! twice, and the remaining-cell count stayed at 3. The repeat guard and count used player.shots instead of enemy.shots.

Commands used: python main.py; inputs: 3,3; 3,3; 1,1; q.

Before changes: actual terminal gameplay

```
BATTLESHIP | BEFORE | actual seeded terminal session

Battleship

Your shots are coordinates like 2,3.
Ship cells remaining: 3
> 3,3
HIT!
AI fired at 4,3

Your shots are coordinates like 2,3.
Ship cells remaining: 3
> 3,3
HIT!
AI fired at 2,4

Your shots are coordinates like 2,3.
Ship cells remaining: 3
> 1,1
MISS!
AI fired at 5,4

Your shots are coordinates like 2,3.
Ship cells remaining: 3
> q
```

Section 2	Section 2 - Implementation of Tasks 1–4
	Task 1 — Coordinates: Every internal coordinate is a zero-based (row, column) tuple. Input converts once from the displayed range 1–6. Output adds one only when printing a coordinate. Both players resolve shots through Board.fire, which validates bounds before changing state. **Task 2 — Fleet and victory:** Each side has a Cruiser (3 cells), Destroyer (2 cells), and Patrol boat (1 cell). Each Ship stores its cells and hits. Board.fire returns hit, miss, or sunk with the ship name. Repeated shots raise an error without advancing the turn. Victory is checked immediately after either side fires; no extra turn follows a winning shot. **Task 3 — AI targeting:** The AI remembers all tried coordinates. After an actual hit, it queues orthogonal neighbors within the board and prefers untried queued cells over random search. Stale candidates are skipped. Sinking a ship clears the queue. When every cell has been tried, choose returns None and the game exits cleanly. **Task 4 — Feedback:** Board and AI operations print nothing. The game formats exactly one HIT!, MISS!, or SUNK! line per accepted shot. Invalid and repeated input receives an error without shot feedback or an AI turn. AI candidate selection does not fire speculative shots. **Module responsibilities:** main.py remains the entry point. board.py owns fleet and shot state; ai.py selects coordinates and consumes results; game.py handles setup, input, turns, and messages. The game remains standard-library-only, with all state in memory. **Setup:** Run python main.py from the repository root. Enter row,col, or q to quit. The fixed fleet layout follows the original teaching scenario; random placement was not a README requirement.

Section 3	

Section 3 - Verification and completed gameplay

Task: Test the README requirements and boundary cases, then capture the completed game.

Automated results: All 21 unittest cases passed. Coverage includes every cell on the 6 × 6 board, hits and misses, repeated hits and misses, separate ship damage, sinking one ship and the full fleet, both victory paths, invalid placements and coordinates, exact AI coordinate handling, adjacent targeting, corner boundaries, stale targets, 36 unique AI shots, exhaustion, silent internal operations, invalid console input, q, EOF, and keyboard interruption.

Commands used: python -m unittest -v (repository root). Equivalent from the lab directory: python -m unittest discover -s 19_battleship -v.

After video: The 10-second recording shows a hit, repeat rejection, a miss, an AI hit followed by an adjacent shot, named ship sinking, decreasing remaining cells, and final player victory after all three enemy ships are sunk.

Recording method: Both videos record output from a running game in a real pseudo-terminal, rendered into terminal frames and encoded as MP4 at 10 frames per second. Seed 7 makes the demonstrations reproducible; ordinary gameplay is not seeded. ffprobe confirmed each video lasts exactly 10.000 seconds.

After changes: final fleet victory

```
BATTLESHIP | AFTER | actual seeded terminal session


Fire at row,col (1-6); q to quit.
Enemy ship cells remaining: 3
Your ship cells remaining: 5
> 1,1
You fired at 1,1: MISS!
AI fired at 3,4: MISS!

Fire at row,col (1-6); q to quit.
Enemy ship cells remaining: 3
Your ship cells remaining: 5
> 5,1
You fired at 5,1: HIT!
AI fired at 2,3: HIT!

Fire at row,col (1-6); q to quit.
Enemy ship cells remaining: 2
Your ship cells remaining: 4
> 5,2
You fired at 5,2: SUNK! (Destroyer)
AI fired at 2,5: MISS!

Fire at row,col (1-6); q to quit.
Enemy ship cells remaining: 1
Your ship cells remaining: 4
> 1,6
You fired at 1,6: SUNK! (Patrol boat)
You sank the fleet. You win!
```

Automated test output: 21 passing cases

```
$ python -m unittest discover -s 19_battleship -v

corner_targets_and_stale_candidates ... ok
hit_prioritizes_untried_neighbor ... ok
no_repeats_and_exhaustion ... ok
selection_and_feedback_are_silent ... ok
size_validation ... ok
sunk_clears_obsolete_targets ... ok
board_operations_are_silent ... ok
empty_board_is_not_a_completed_game ... ok
every_cell_hit_or_miss ... ok
individual_damage_and_complete_fleet ... ok
invalid_coordinates_do_not_consume_shots ... ok
invalid_ship_placements_are_atomic ... ok
no_placement_after_shooting ... ok
repeated_hit_and_miss_do_not_mutate_state ... ok
ai_exhaustion_is_graceful ... ok
ai_hit_uses_exact_chosen_coordinate ... ok
ai_win_stops_game ... ok
eof_and_keyboard_interrupt_quit ... ok
invalid_input_and_quit_do_not_advance_turn ... ok
player_coordinates_repeat_and_one_feedback_per_shot ... ok
player_win_stops_before_extra_ai_turn ... ok
Ran 21 tests in 0.004s
OK
```

Section 4	Section 4 - Submission files and LLM use
	Repository: https://github.com/DevaOnL/se-lab04-19-battleship
	Deliverables: Lab-4/Updated_Code contains the submission snapshot; the same working modules remain at the repository root for easy execution. Lab-4/Videos contains before_gameplay.mp4 and after_gameplay.mp4. Lab-4/Evidence contains test output, gameplay transcripts, and the original coordinate-defect trace. This report is supplied in DOCX and PDF.
	Chat history: The accompanying PES1UG24CS145_SE_Lab4_Chat_History.docx and .pdf export the available user and assistant conversation through the documented export cutoff. The GitHub-hosted export is linked in the submission README. A separate platform share URL has not been supplied.
	LLM assistance: Codex inspected the original code, reproduced defects, implemented the four tasks, added regression tests, captured gameplay, and prepared this report. The regression suite exercises observable behavior; a failing test double was corrected to preserve the real AI selection path. This report records the work performed and does not assert that the student independently performed these steps.
	Report format: The report follows Assignment_Helper_FINAL.zip: the supplied university logo in the header, centered title block, student metadata, and one four-column master table. Student details match earlier SE submissions. Course code UE24CS342AB2 follows Labs 1 and 2; the Lab 3 report used a different code.
	Student: Devansh Gupta — PES1UG24CS145 — Section C.